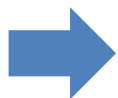


学习内容

CONTENTS



1

机器学习概述

2

线性回归与逻辑回归

3

支持向量机

4

聚类与降维

5

决策树

6

强化学习

- 线性回归
- 逻辑回归
- **SVM**（支持向量机）
- 决策树
- K-means聚类
- 强化学习

监督学习

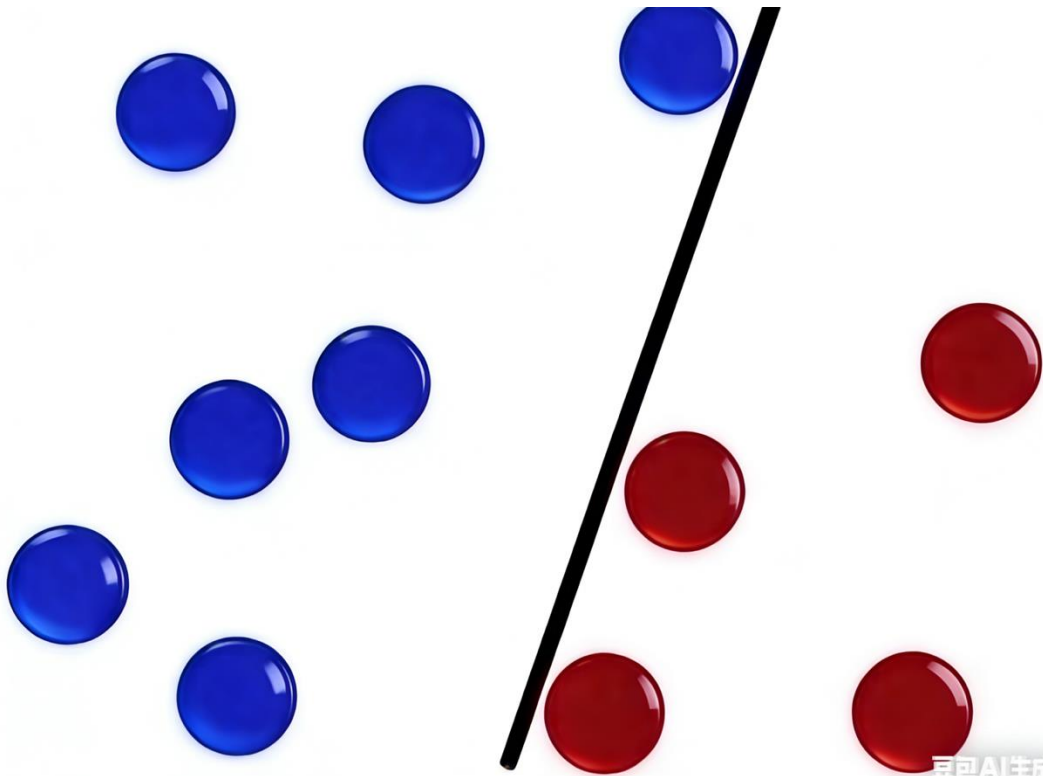
numpy: python第三方库, 用于**科学计算**
matplotlib:python第三方库, 主要用于进行**可视化**
sklearn:python的重要机器学习库, 其中封装了大量的**机器学习算法**, 如: **分类、回归、降维以及聚类**

支持向量机(Support Vector Machine): Cortes和Vapnik于**1995**年首先提出的, 在解决**小样本、非线性及高维模式识别**中表现出许多特有的优势, 并能够推广应用到函数拟合等其他机器学习问题中。

在很久以前的情人节，大侠要去救他的爱人，但魔鬼和他玩了一个游戏。

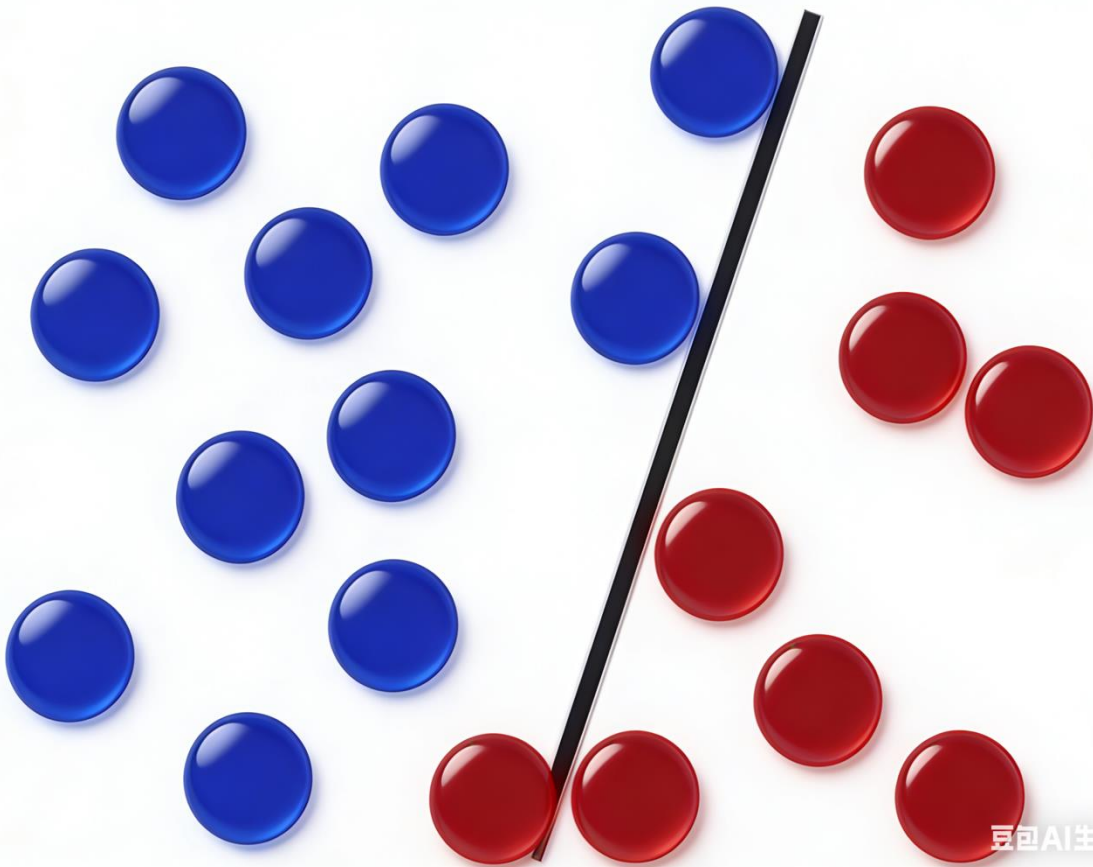
1. 魔鬼在桌子上似乎有规律放了两种颜色的球，说：“你用一根棍分开它们？要求：尽量在放更多球之后，仍然适用。”

于是大侠这样放，干的似乎还不错？



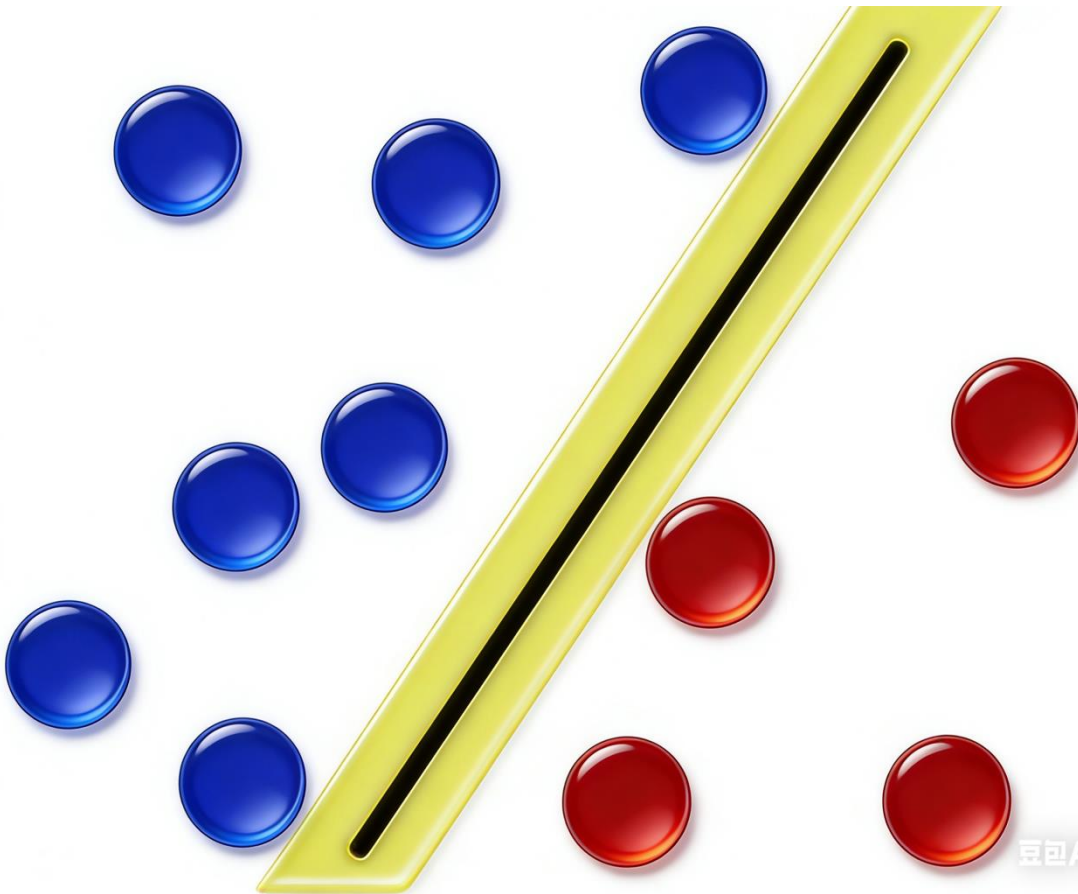
2. 然后魔鬼，又在桌上放了更多的球。

似乎有一个球站错了阵营。

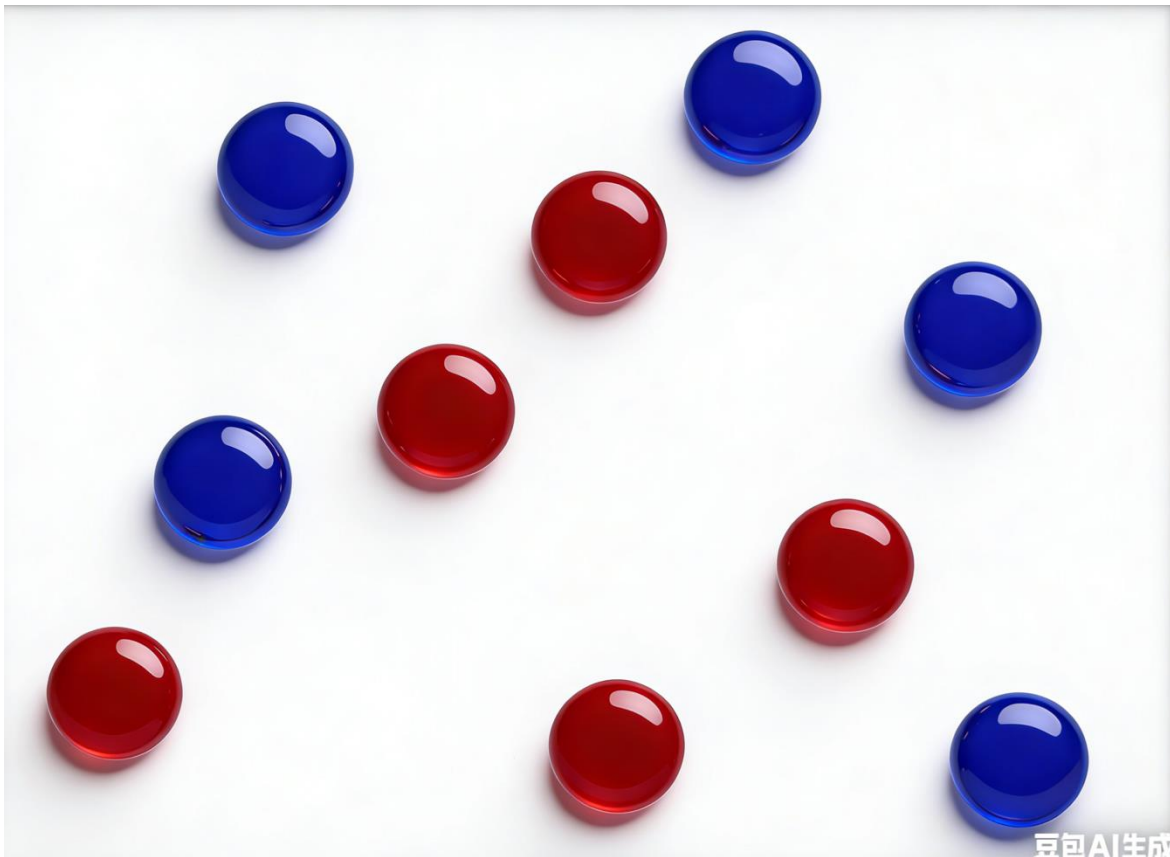


3. SVM就是试图：

把棍放在最佳位置，
好让在棍的两边有尽
可能大的间隙。

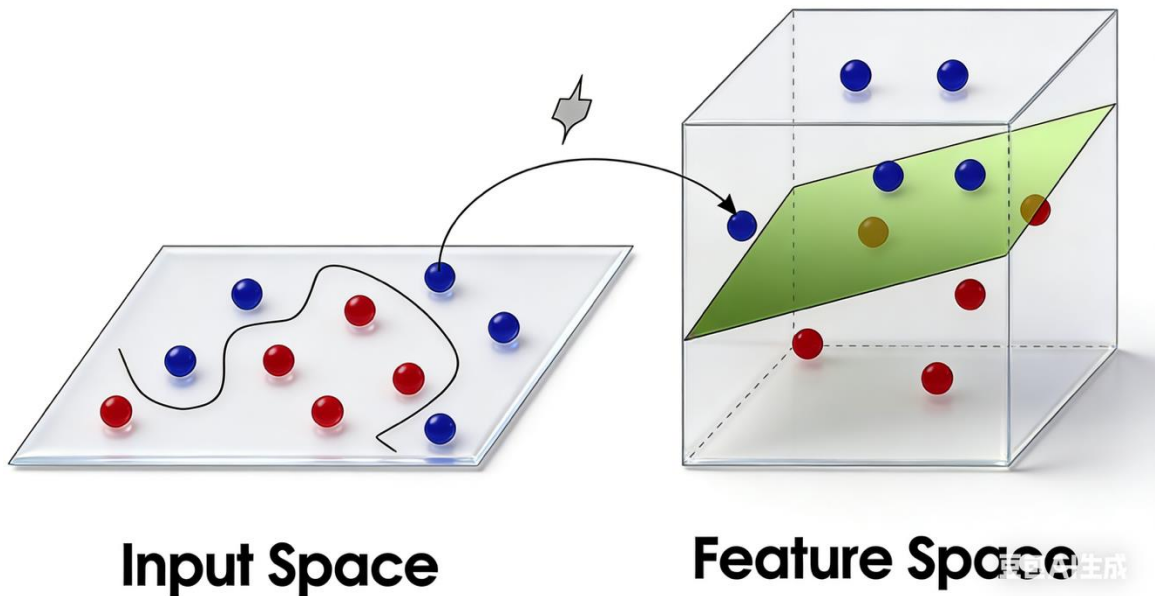


4. 魔鬼看到大侠已经学会了一个trick, 于是魔鬼给了大侠一个新的挑战。



5. 现在，大侠没有棍可以很好帮他分开两种球了，怎么办呢？

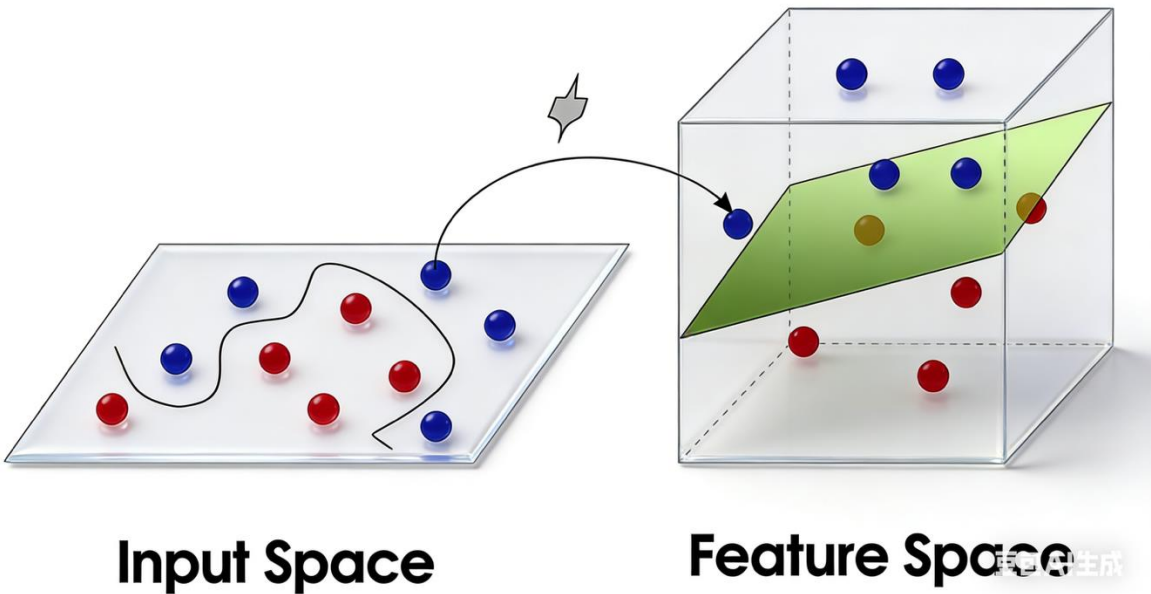
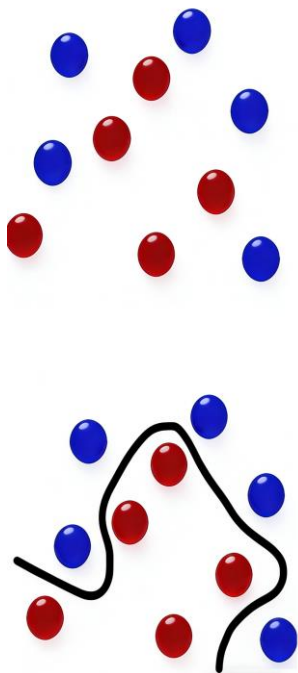
大侠**桌子一拍**，球飞到空中。然后，凭借大侠的轻功，抓起**一张纸**，插到了两种颜色的**球**的**中间**。



5. 现在，大侠没有棍可以很好帮他分开两种球了，怎么办呢？

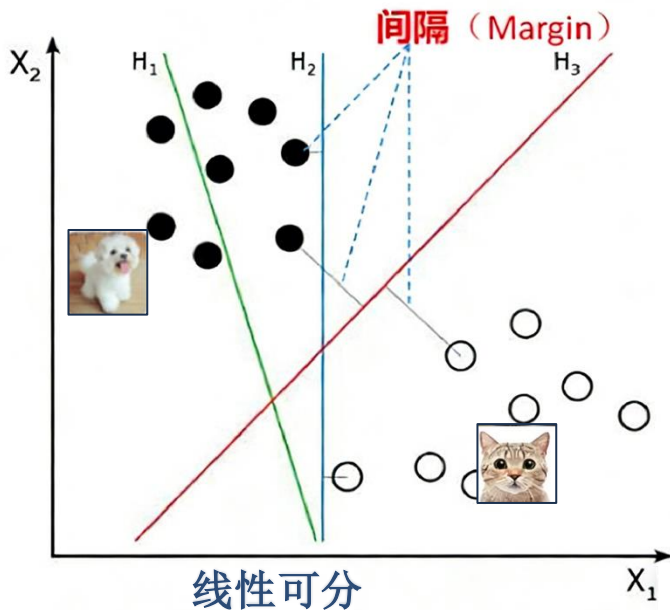
大侠**桌子一拍**，球飞到空中。然后，凭借大侠的轻功，抓起**一张纸**，插到了两种颜色的**球**的**中间**。



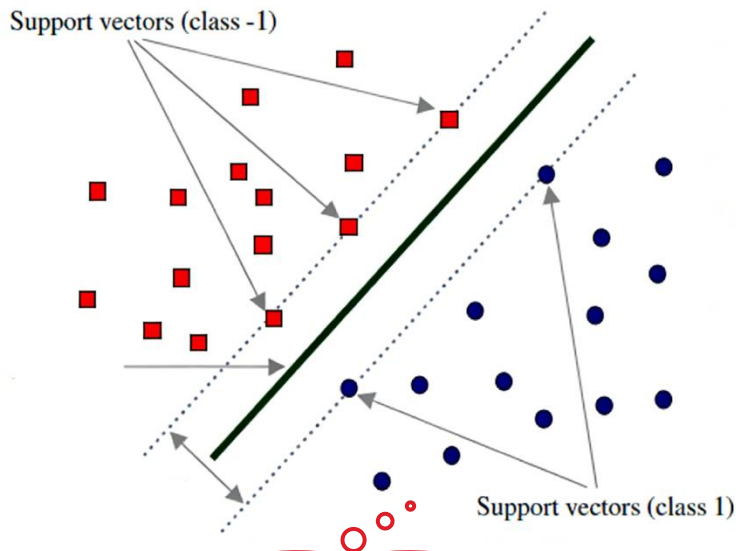


球: **data**, 最大间隙 **trick: optimization**, 拍桌子: **kernelling**
棍子: **classifier**, 那张纸: **hyperplane**

直观理解： 对一个数据点进行分类，当超平面离数据点的“间隔”越大，分类的确信度 (confidence) 也越大。



支持向量机： 中间直线就是分类超平面，不难发现，用来确定这条直线其实只需要两条虚线上的五个点就够了。

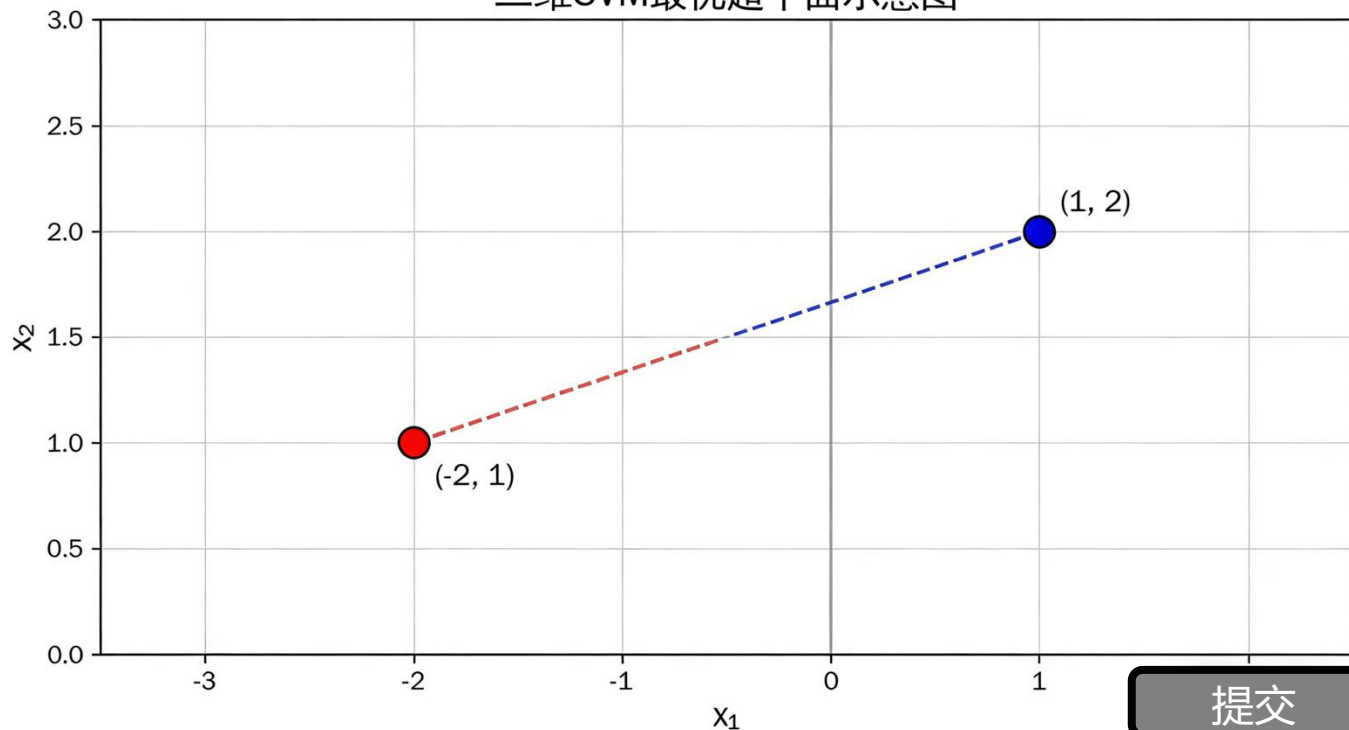


如何从茫茫样本中
找出支持向量？

计算题 5分

问题：在二维空间上，假定一个正例为 $(-2, 1)$ ，一个负例为 $(1, 2)$ ，请计算支持向量机超平面。

二维SVM最优超平面示意图



提交

计算题 5分



问题：在二维空间上，假定一个正例为 $(-2, 1)$ ，一个负例为 $(1, 2)$ ，请计算支持向量机超平面。

1. 方法一

(a) 超平面为A（正例）、B（负例）两点连线的中垂线。

(b) A、B两点的中点为 $(-\frac{1}{2}, \frac{3}{2})$ 。

(c) 采用直线方程 $(y=kx+b)$

i. 中垂线斜率为 $K = -\frac{1}{\frac{2-1}{1-(-2)}} = 3$ 。

ii. 中垂线的直线方程 $(y = kx + b)$ 为：

$$\begin{aligned} \square y - \frac{3}{2} &= -3[x - (-\frac{1}{2})] \\ \Rightarrow y - \frac{3}{2} &= -3x - \frac{3}{2} \\ \Rightarrow y &= -3x \end{aligned}$$

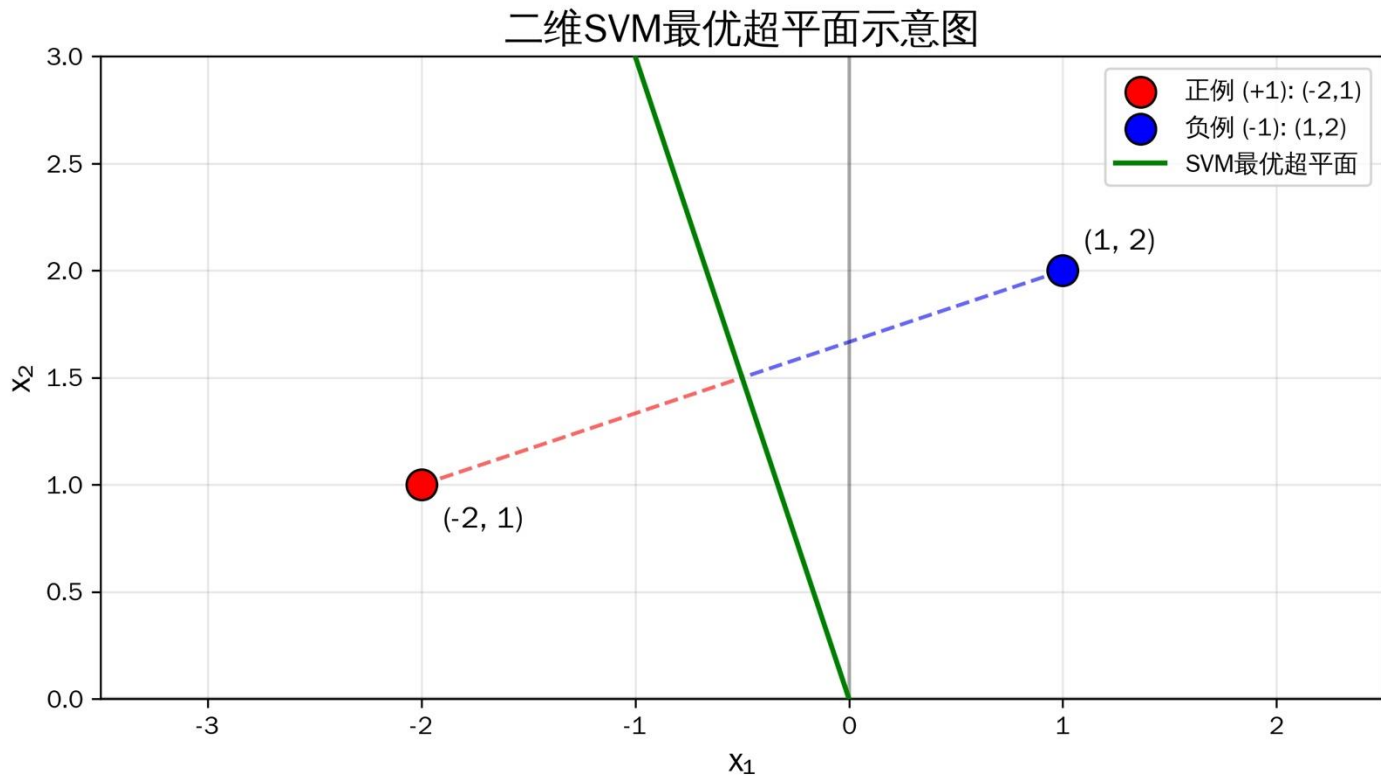
(d) 或采用直线方程 $(ax + by + c = 0)$

i. AB之间的矢量 $\overrightarrow{AB} = (1, 2) - (-2, 1) = (3, 1)$ 。中垂线的方程即为 $3x + y + c = 0$ 。

ii. 代入中点 $(-\frac{1}{2}, \frac{3}{2})$ ，得其方程为 $3x + y = 0$ 。

计算题 5分

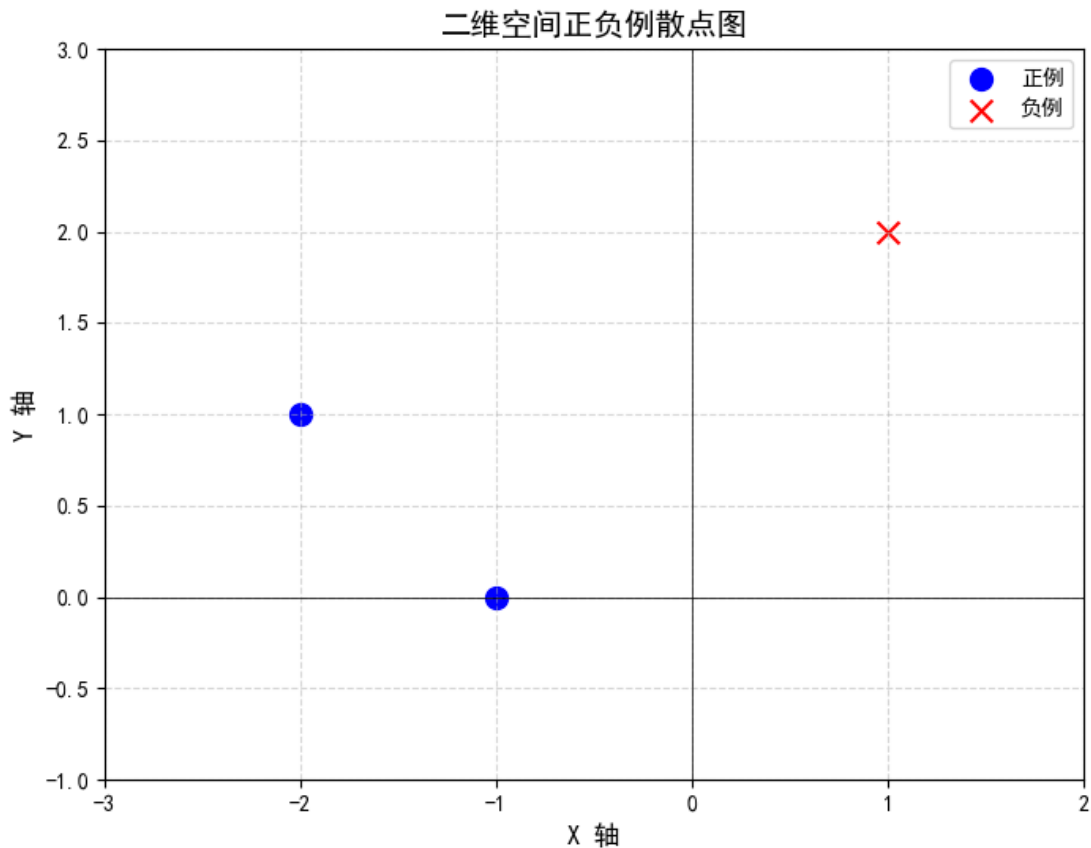
问题：在二维空间上，假定一个正例为 $(-2, 1)$ ，一个负例为 $(1, 2)$ ，请计算支持向量机超平面。



思考



三个数据点，如何确定支持向量呢？正例： $(-2, 1)$, $(-1, 0)$ ；负例 $(1, 2)$ 。

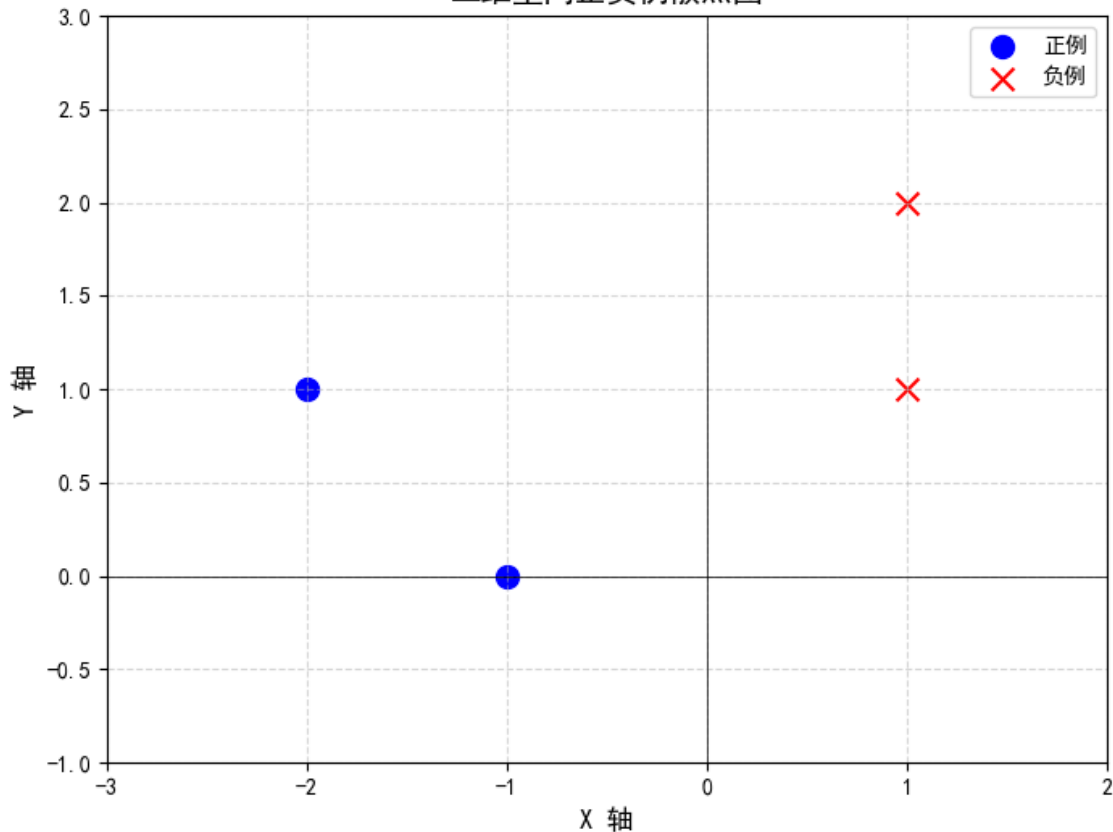


思考



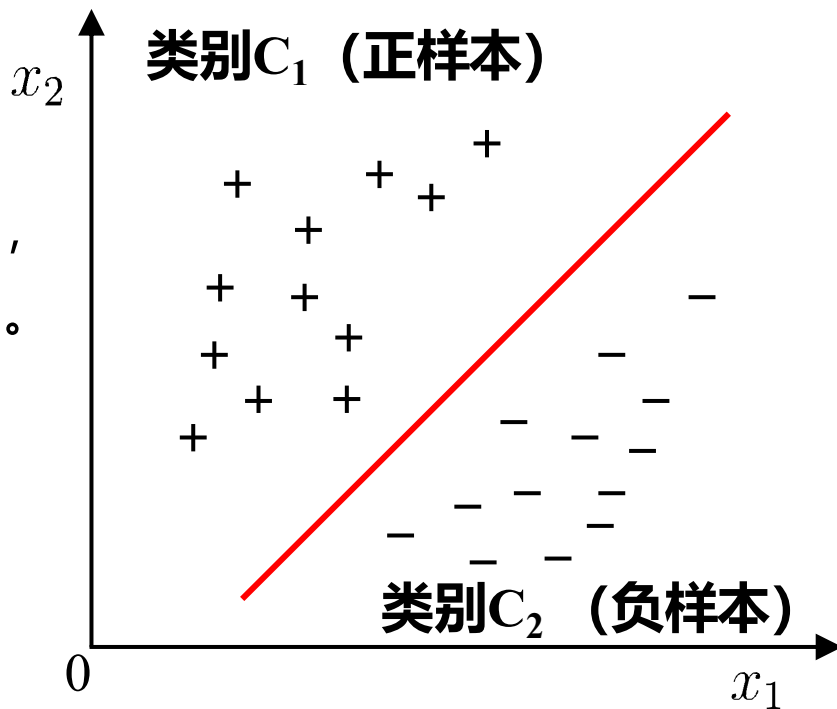
四个数据点呢？正例： $(-2, 1)$, $(-1, 0)$ ；负例 $(1, 2)$, $(1, 1)$ 。

二维空间正负例散点图



- 定义一个**线性函数**: $g(x) = w^T x + b$
 w : 参数向量, 决定了超平面的方向,
 b : 偏置, 决定了超平面与原点之间的距离。
 - 若 $g(x_i) > 0$, 就判别为**类别C₁** ($y = 1$),
 - 若 $g(x_i) < 0$, 则判别为**类别C₂** ($y = -1$)。
- 等价于给 $g(x)$ 加一个**符号函数** $\text{sign}()$, 即
 $f(x) = \text{sign}(g(x))$ 是真正的分类决策函数。

红色直线 (分界线) 的表达式是 $g(x) = 0$,
即 $w^T x + b = w_1 x_1 + w_2 x_2 + b = 0$, 这个函数叫做**分类函数**。

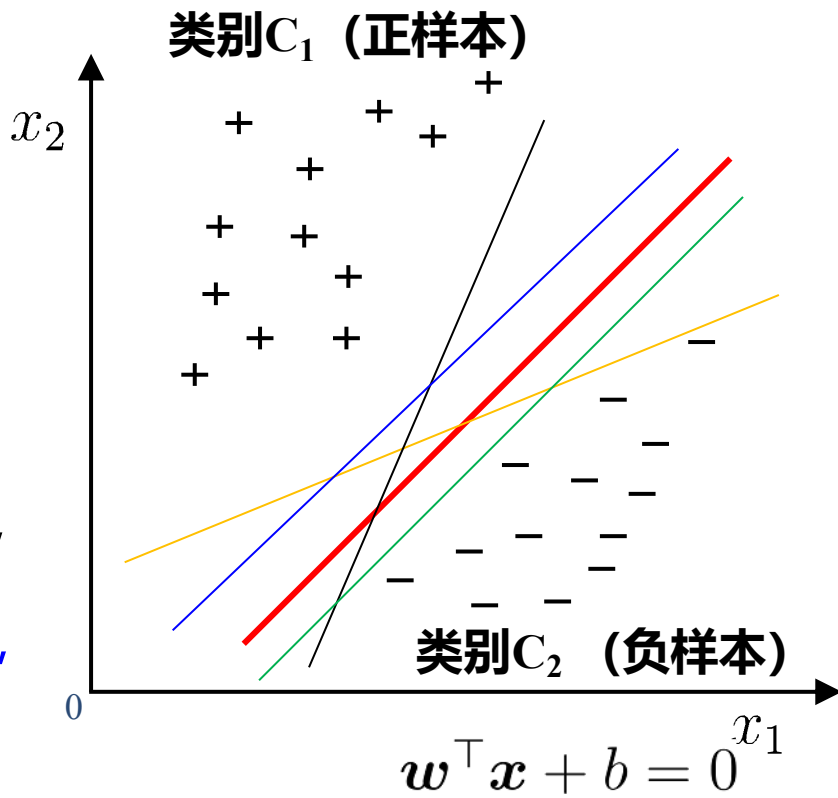


红色直线 (分界线)的表达式是 $g(x) = 0$, 即 $w^T x + b = w_1 x_1 + w_2 x_2 + b = 0$, 这个函数叫做**分类函数**。

分界线并不是唯一的。

对同一个问题存在多个分类函数的时候, 哪一个函数更好呢?

量化 “好” 的程度, 通常叫 “分类间隔”



2.3 支持向量机



- 给定样本集: $D=\{(x_1,y_1),(x_2,y_2),\dots,(x_N,y_N)\}$, $y_i \in \{-1,1\}$

可以定义一个样本点 x_i 到某个超平面的几何间隔 γ_i :

回忆解析几何, 点 (x,y) 到直线 $Ax+By+C=0$ 的距离:

$$\frac{|Ax+By+C|}{\sqrt{A^2+B^2}}$$

$$\gamma_i = \frac{|w^T x_i + b|}{\|w\|} = \frac{y_i(w^T x_i + b)}{\|w\|}$$

函数间隔

支持向量机的目标是寻找一个超平面 (w^*, b^*) 使得 γ 最大, 即

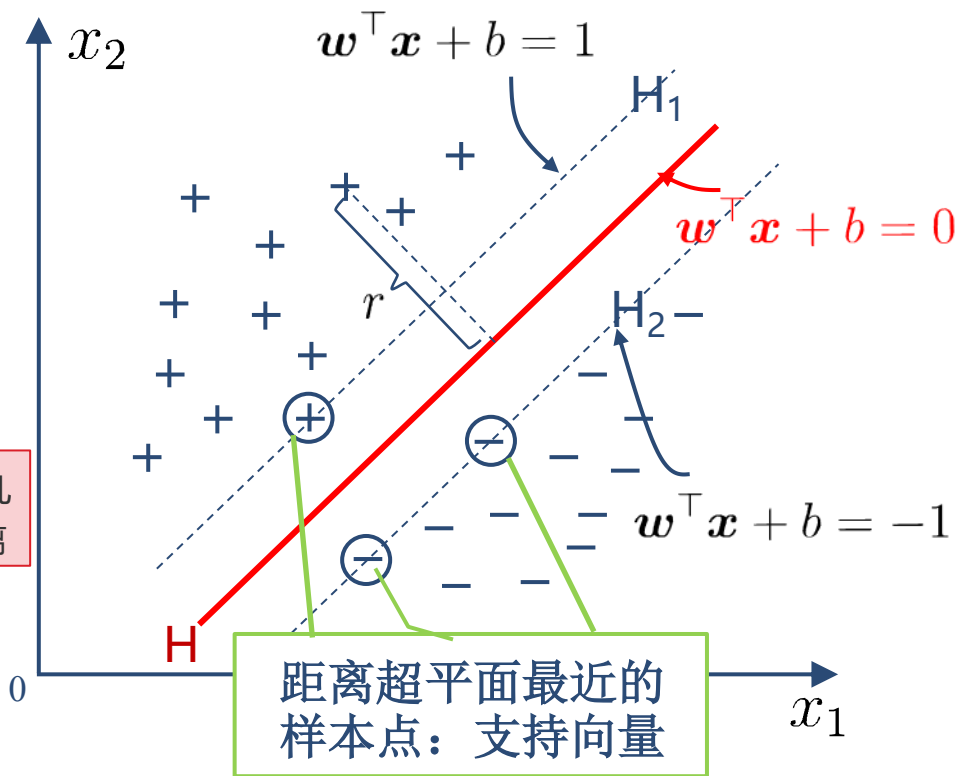
$$\max_{w,b} \gamma \quad \gamma = \min_{i=1,2,\dots,N} \gamma_i$$

超平面关于所有样本点的几何间隔的最小值, 即支持向量到超平面的距离

$$\text{s.t.} \quad \frac{y^{(n)}(w^T x^{(n)} + b)}{\|w\|} \geq \gamma, \forall n \in \{1, \dots, N\}$$

• 上面公式中 s.t 是 subject to 的缩写, 也就是限制条件的意思。

$$\|w\| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$



支持向量机的目标是寻找一个超平面 $(\mathbf{w}^*, b^*)$ 使得 γ 最大, 即

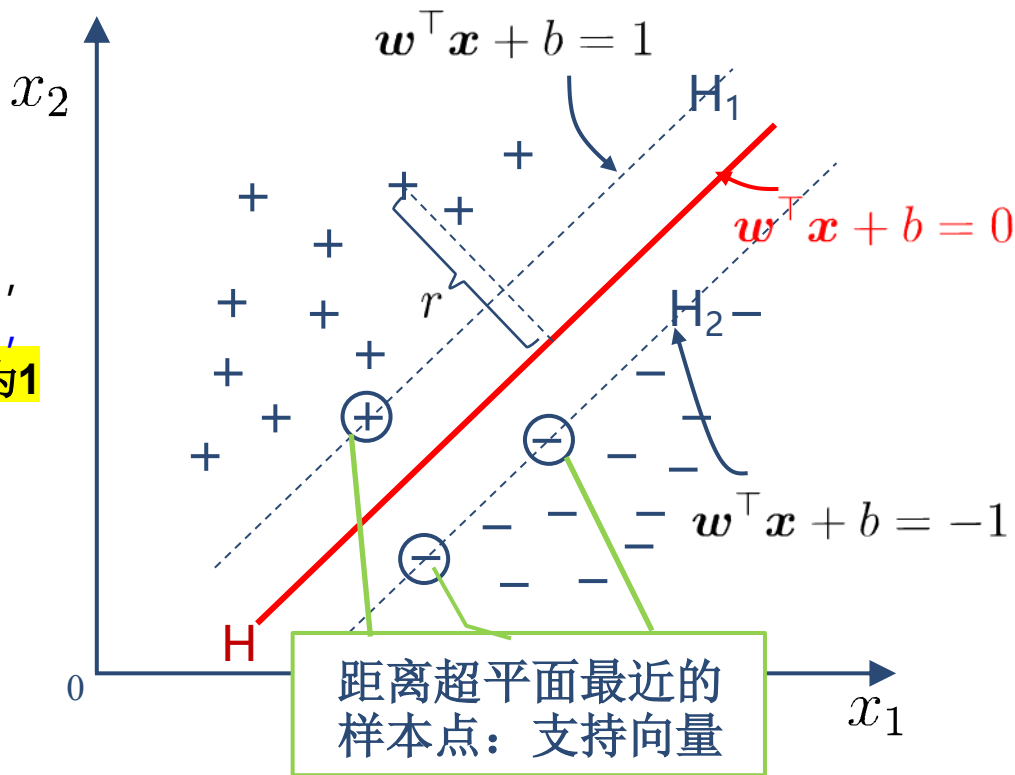
$$\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \gamma \\ \text{s.t.} \quad & \frac{y^{(n)}(\mathbf{w}^\top \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|} \geq \gamma, \forall n \in \{1, \dots, N\} \end{aligned}$$

函数间隔

假设 H 是分类面, 那么 H_1 和 H_2 是平行于 H , 且过离 H 最近的两类样本的直线, H_1 与 H , H_2 与 H 之间的距离就是 $1/\|\mathbf{w}\|$, 函数间隔为 1

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \frac{1}{\|\mathbf{w}\|^2} \iff \min \|\mathbf{w}\| \iff \min \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \end{aligned}$$



硬间隔支持向量机：（线性可分）

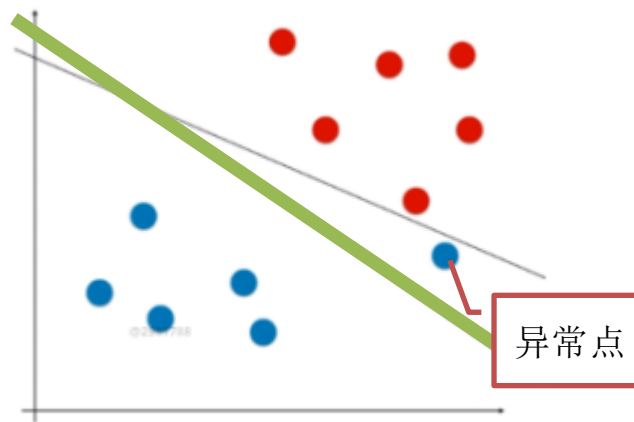
$$\min \frac{1}{2} \|\mathbf{w}\|_2^2$$

$$s.t. \ y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1$$

支持向量机（SVM）
的基本模型

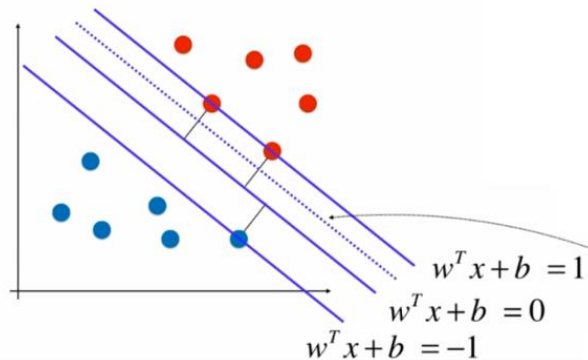
求解上式得到大间隔划分超平面对应的模型
 $g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ ，其中 $\mathbf{w}$ 和 b 是模型参数。

$$\|\mathbf{w}\|^2 = w_1^2 + w_2^2 + \dots + w_n^2$$



软间隔支持向量机：（近似线性可分）

惩罚
参数



L1正则

$$\min \frac{1}{2} \|\mathbf{w}\|_2^2 + \boxed{C} \sum_{i=1}^m \zeta_i$$

$$s.t. \ y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \zeta_i$$

$$\zeta_i \geq 0$$

L2正则

$$\min \frac{1}{2} \|\mathbf{w}\|_2^2 + \boxed{C} \sum_{i=1}^m \zeta_i^2$$

$$s.t. \ y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \zeta_i$$

$$\zeta_i \geq 0$$

$$(\mathbf{w}^T \mathbf{x} + b) = 1 - \boxed{\zeta_i}$$

松弛
变量

软间隔：允许支持向量机在一些样本上不满足硬约束，避免过拟合

方案：还是使用直线，不过不追求完全可分，会适当包容一些分错的情况，会在模型中加入惩罚函数，尽量让分错的点不要太多太离谱。对分错点的惩罚函数就是这个点到其正确位置的距离。

- 惩罚系数C很大的时候，分错的点就会更少，但是过拟合的情况可能会比较严重。
- 惩罚系数C很小的时候，分错的点可能会很多，不过可能由此得到的模型也会不太正确。

<https://www.cnblogs.com/keye/p/10633530.html>

- 拉格朗日乘子法(以硬间隔SVM为例)

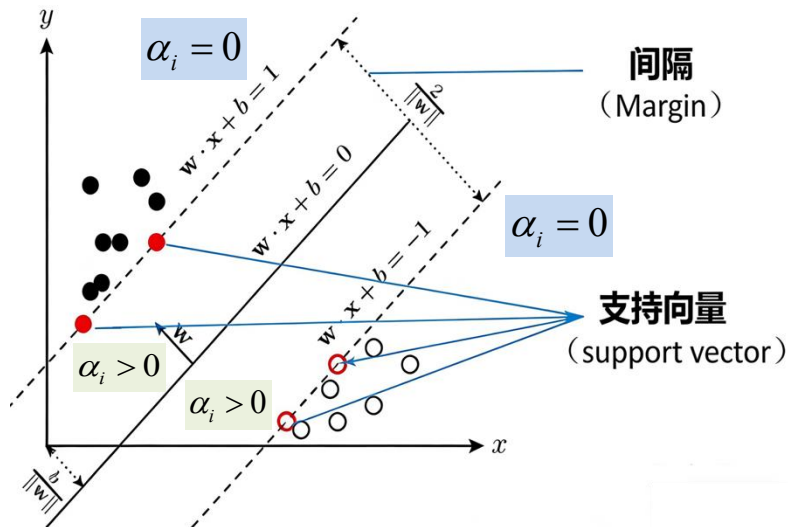
$$\min_{w, b} \frac{1}{2} \|w\|^2$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1, \quad i = 1, 2, \dots, m.$$

- 第一步：引入拉格朗日乘子 $\alpha_i \geq 0$
得到拉格朗日函数：

(约束条件 $1 - y_i(w^T x_i + b) \leq 0$)

$$\max_{\alpha \geq 0} \min_{w, b} L(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^N \alpha_i (1 - y_i(w^T x_i + b))$$



<https://www.cnblogs.com/keye/p/10633530.html>

- 拉格朗日乘子法(以硬间隔SVM为例)

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1, \quad i = 1, 2, \dots, m.$$

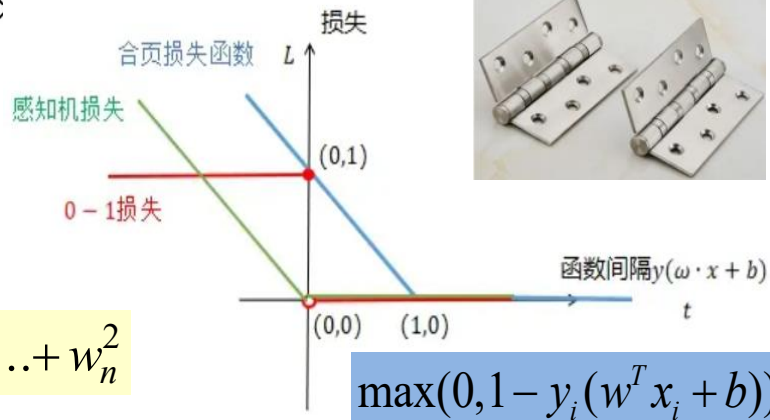
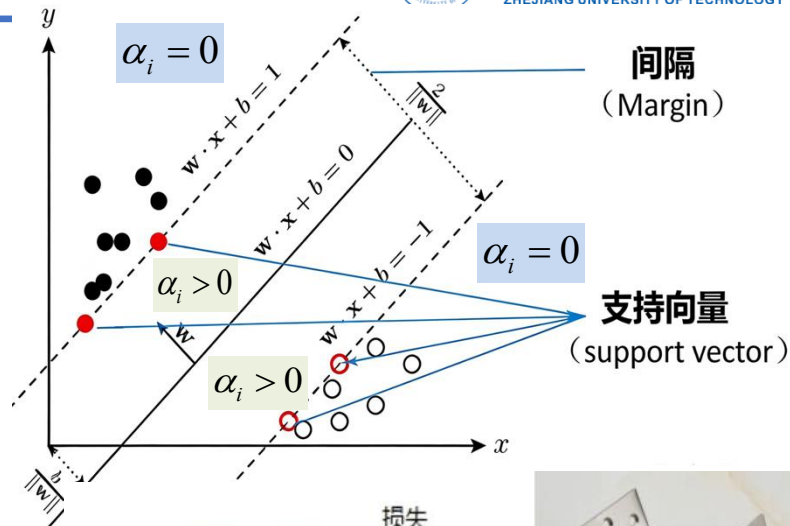
— 第一步: 引入拉格朗日乘子 $\alpha_i \geq 0$

得到拉格朗日函数:

$$\max_{\alpha \geq 0} \min_{w,b} L(w,b,\alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^N \alpha_i (1 - y_i(w^T x_i + b))$$

等价为一个含有正则化的所有样本的合页损失和最小化问题

$$\|w\|^2 = w_1^2 + w_2^2 + \dots + w_n^2$$



三个损失函数的比较

2.3

支持向量机 —— 对偶问题

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

$$\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w}$$

$$\text{s.t. } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m.$$

- 拉格朗日乘子法(以硬间隔SVM为例)

- 第一步: 引入拉格朗日乘子 $\alpha_i \geq 0$ 得到拉格朗日函数

$$\max_{\alpha \geq 0} \min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

- 第二步: 令 $L(\mathbf{w}, b, \alpha)$ 对 $\mathbf{w}$ 和 b 的偏导为零可得

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i, \quad \sum_{i=1}^m \alpha_i y_i = 0.$$

- 第三步: 回代并转成求极小, 得到等价的对偶最优化问题:

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^m \alpha_i$$

$$\text{s.t. } \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m.$$

拉格朗日函数

$$\begin{aligned}
 L(w, b, \alpha) &= \frac{1}{2} w^T w + \sum_{i=1}^N \alpha_i (1 - y_i (w^T x_i + b)) \\
 &= \frac{1}{2} w^T w - \sum_{i=1}^N \alpha_i y_i (w^T x_i + b) + \sum_{i=1}^N \alpha_i
 \end{aligned}$$

求 $L(w, b, \alpha)$ 对 w, b 的极小值

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^N \alpha_i y_i x_i = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i$$

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^N \alpha_i y_i = 0 \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0$$

$$\begin{aligned}
 \max_{\alpha} \min_{w, b} L(w, b, \alpha) &= \frac{1}{2} \left(\sum_{i=1}^N \alpha_i y_i x_i \right)^T \sum_{i=1}^N \alpha_i y_i x_i - \sum_{i=1}^N \alpha_i y_i \left(\left(\sum_{i=1}^N \alpha_i y_i x_i \right)^T x_i + b \right) + \sum_{i=1}^N \alpha_i \\
 &= \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i y_i \alpha_j y_j x_i^T x_j - \sum_{i=1}^N \sum_{j=1}^N \alpha_i y_i \alpha_j y_j x_j^T x_i + \sum_{i=1}^N \alpha_i \\
 &= - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i y_i \alpha_j y_j x_i^T x_j + \sum_{i=1}^N \alpha_i
 \end{aligned}$$

$$\max_{\alpha} -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i y_i \alpha_j y_j x_i^T x_j + \sum_{i=1}^N \alpha_i$$

$$\text{s.t.} \quad \sum_{i=1}^N \alpha_i y_i = 0$$

$$\alpha_i \geq 0, i = 1, 2 \dots N$$

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i y_i \alpha_j y_j x_i^T x_j - \sum_{i=1}^N \alpha_i$$

$$\text{s.t.} \quad \sum_{i=1}^N \alpha_i y_i = 0$$

$$\alpha_i \geq 0, i = 1, 2 \dots N$$

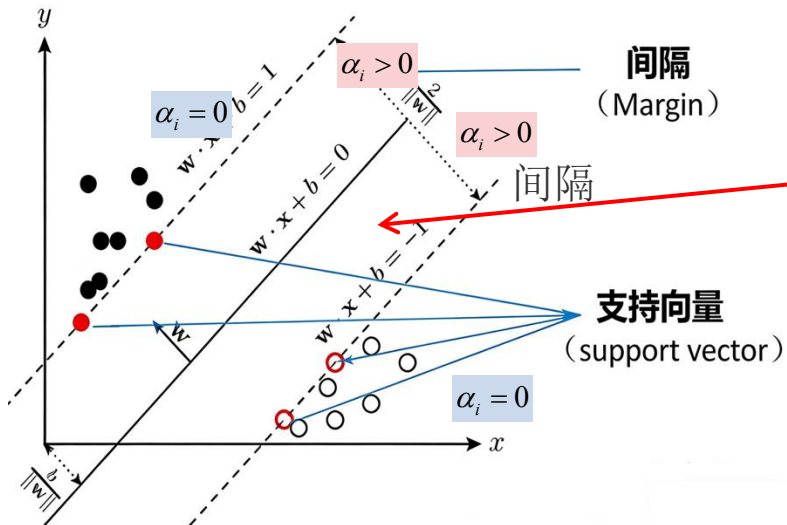
求 $L(w, b, a)$ 对 α 的极大值

可用序列最小最优化
(SMO) 算法求得 α^*

$$b^* = y_j - \sum_{i=1}^N \alpha_i^* y_i x_i^T x_j$$

$$w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$$

$$y_j (w^T x_j + b) = 1$$



线性可分

$$\begin{cases} w^T x_i + b \geq +1, & y_i = +1 \\ w^T x_i + b \leq -1, & y_i = -1 \end{cases}$$

虚线：支持向量所在的边界

分类超平面（决策边界）模型：

$$g(x) = w^T x + b = \sum_{i=1}^N \alpha_i^* y_i x_i^T x_j + b = 0$$

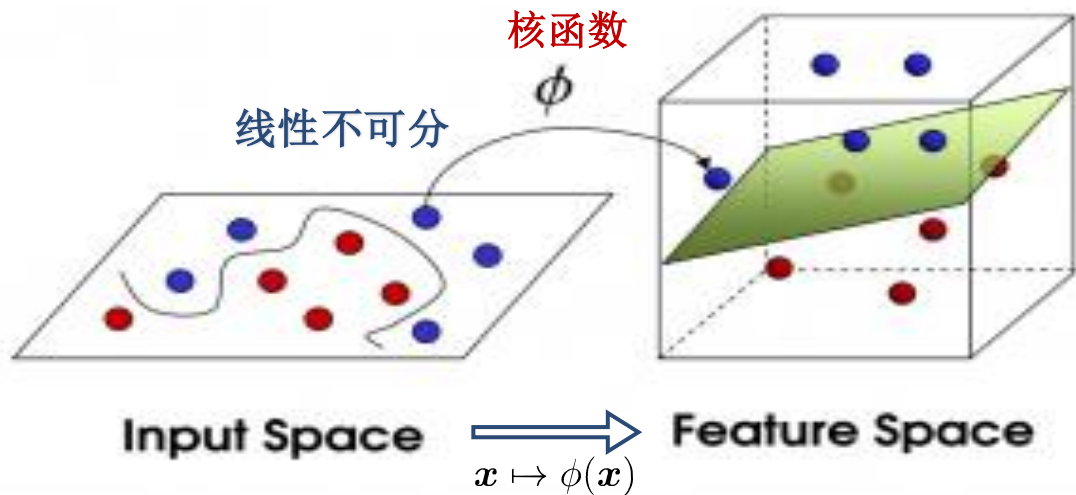
$$w^* = \sum_{i=1}^N \alpha_i^* y_i x_i \quad b^* = y_j - \sum_{i=1}^N \alpha_i^* y_i x_i^T x_j$$

w , b 跟所有支持向量的特征 x_i 和类别 y_i 有关

- 对于线性可分的数据集来说，分类超平面可能多个，但几何间隔最大的分类超平面是唯一的。
- 支持向量机解的稀疏性：训练完成后，大部分的训练样本都不需保留，最终模型仅与支持向量有关。

用曲线去将其完全分开, 即非线性的决策边界。

线性可分



核函数: 直接计算两个向量在高维特征空间中的内积, 而无需显式地计算出高维映射本身。这避免了“维数灾难”, 计算成本大大降低

$$g(x) = w^T x + b = \sum_{i=1}^m \alpha_i y_i x_i^T x + b$$

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

$$b = y_j - \sum_{i=1}^m \alpha_i y_i x_i^T x_j$$

$$g(x) = w^T \phi(x) + b = \sum_{i=1}^m \alpha_i y_i \phi(x_i)^T \phi(x) + b$$

核函数 $\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$

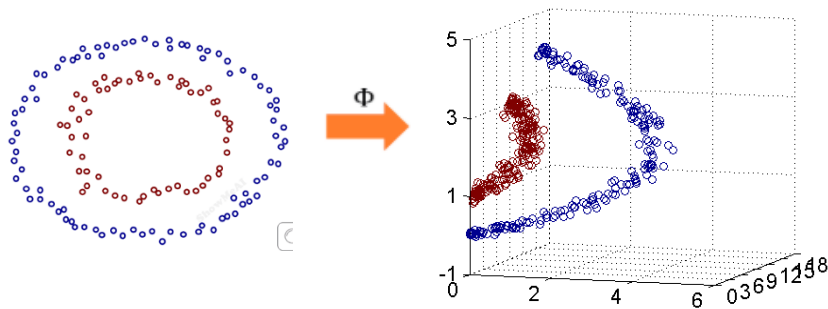
► 高维空间中的线性分类器

$$g(x) = w^T \phi(x) + b = \sum_{i=1}^m \alpha_i y_i \phi(x_i)^T \phi(x) + b$$

核函数 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$

名称	表达式	参数
线性核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$	
多项式核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j)^d$	$d \geq 1$ 为多项式的次数
高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\delta^2}\right)$	$\delta > 0$ 为高斯核的带宽(width)
拉普拉斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ }{\delta}\right)$	$\delta > 0$
Sigmoid核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^T \mathbf{x}_j + \theta)$	$\tanh$ 为双曲正切函数, $\beta > 0, \theta < 0$

低维空间非线性可分，而高维空间线性可分



映射函数 $z_1 = x_1^2, z_2 = x_2^2, z_3 = x_2$

对映射后的坐标加以旋转, 就可得到一个线性可分的点集

一个本来线性不可分的分类问题，通过核函数映射到高维空间而变成了线性可分的。

2.3

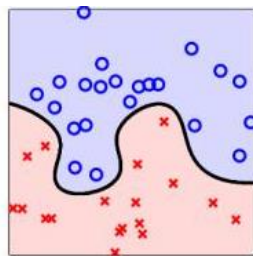
支持向量机：核函数

► 高维空间中的线性分类器

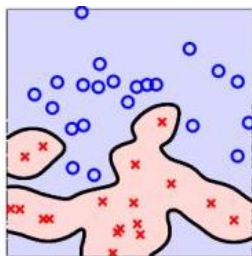
$$g(x) = w^T \phi(x) + b = \sum_{i=1}^m \alpha_i y_i \phi(x_i)^T \phi(x) + b$$

核函数 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$

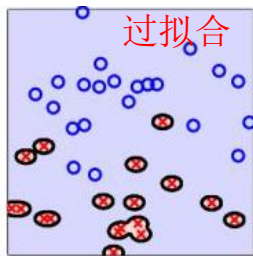
名称	表达式	参数
线性核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$	
多项式核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j)^d$	$d \geq 1$ 为多项式的次数
高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\delta^2}\right)$	$\delta > 0$ 为高斯核的带宽(width)



$\exp(-1 \|\mathbf{x} - \mathbf{x}'\|^2)$

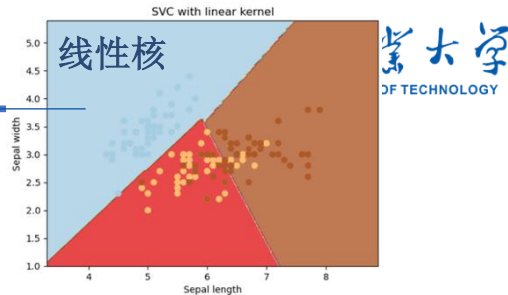


$\exp(-10 \|\mathbf{x} - \mathbf{x}'\|^2)$

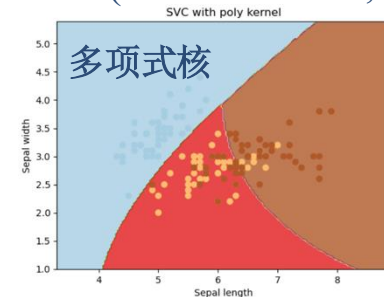


$\exp(-100 \|\mathbf{x} - \mathbf{x}'\|^2)$

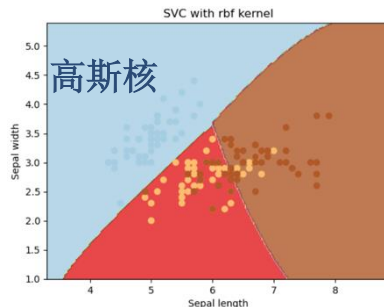
gamma 值越大，会越准确地划分每一个训练样本，会导致泛化误差大



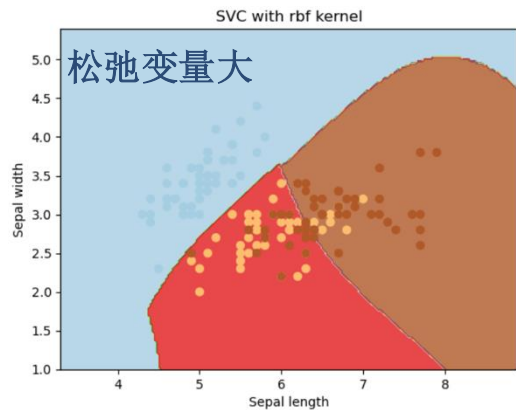
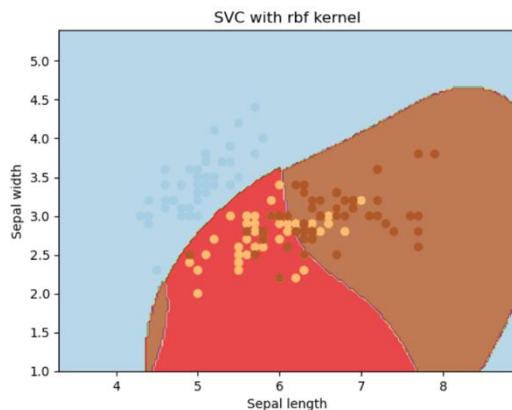
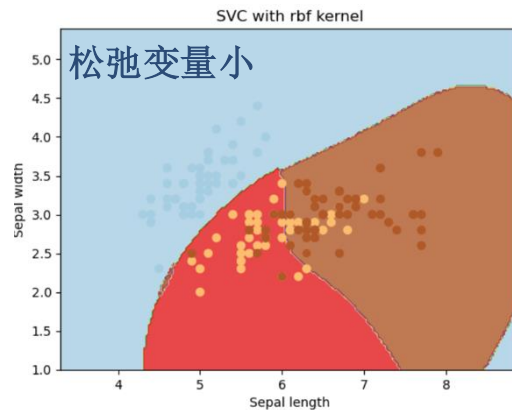
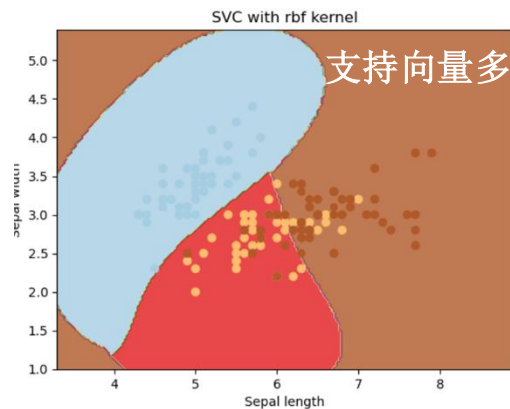
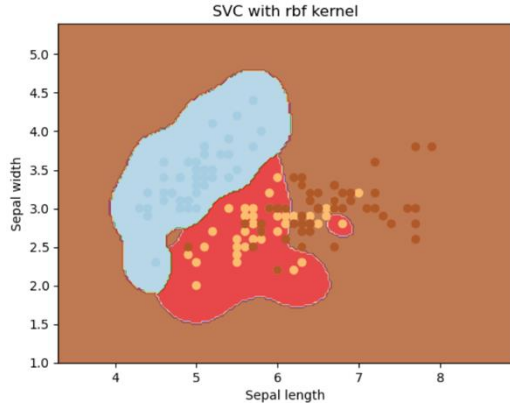
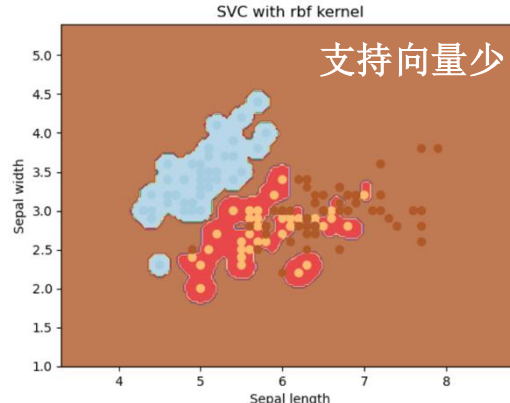
`svm.SVC(kernel='linear', C=1)`



`svm.SVC(kernel='poly', degree=3)`

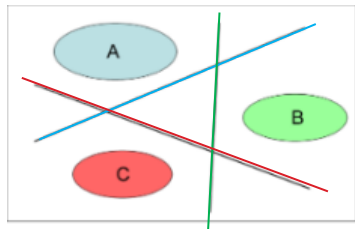


`svm.SVC(kernel='rbf', C=1)`

 $C=10$  $C=100$  $C=1000$  $\gamma=1$  $\gamma=10$  $\gamma=100$

一对多法 (one-versus-rest, 简称OVR SVMs)

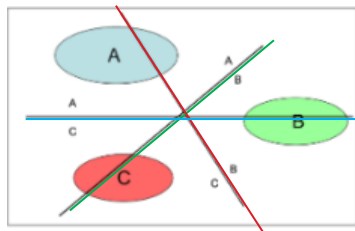
训练时依次把某个类别的样本归为一类,其他剩余的样本归为另一类,这样 k 个类别的样本就构造出了 k 个SVM。分类时将未知样本分类为具有最大分类函数值的那类。



一对一法 (one-versus-one, 简称OVO SVMs或者pairwise)

其做法是在任意两类样本之间设计一个SVM, 因此 k 个类别的样本就需要设计 $k(k-1)/2$ 个SVM。

当对一个未知样本进行分类时, 最后得票最多的类别即为该未知样本的类别。



层次支持向量机

层次分类法首先将所有类别分成两个子类, 再将子类进一步划分成两个次级子类, 如此循环, 直到得到一个单独的类别为止。

- ✓ **Pima Indians Diabetes Data Set**（皮马印第安人糖尿病数据集）：
包含768个样本，该数据集最初来自美国糖尿病/消化/肾脏疾病研究所，旨在基于数据集中包含的某些诊断测量来预测患者是否患有糖尿病。
- ✓ 数据集特点
 - » 样本选择条件：所有样本均为至少21岁的Pima印第安女性。
 - » 数据结构：数据集由八个医学预测变量和一个目标变量组成。预测变量包括患者的怀孕次数、BMI、胰岛素水平、年龄等。目标变量为“Outcome”，表示患者是否患有糖尿病（1表示患有，0表示未患有）。
 - » 数据内容：数据集的内容是Pima印第安人的医疗记录，涵盖了过去5年内是否有糖尿病的信息。所有数据均为数字，且问题是二分类问题。

Pima Indians Diabetes Data Set（皮马印第安人糖尿病数据集） 根据现有的医疗信息预测5年内皮马印第安人糖尿病发作的概率，包含768个样本，旨在基于数据集中包含的某些诊断测量来预测患者是否患有糖尿病。

1.准备数据

```
import numpy as np
import urllib
#糖尿病数据集: https://gitcode.com/open-source-toolkit/282dc
data_path='.\\pima-indians-diabetes.csv'
## 加载CSV文件
dataset = np.loadtxt(data_path, delimiter=",")
## 区分特征和标签
X = dataset[:,0:7]
y = dataset[:,8]
```

2.数据预处理

```
from sklearn import preprocessing
```

```
## 幅度缩放
```

```
scaled_X = preprocessing.minmax_scale(X, feature_range=(0, 1))
```

```
## 归一化
```

```
normalized_X = preprocessing.normalize(X)
```

```
## 标准化
```

```
standardized_X = preprocessing.scale(X)
```

3.特征工程

```
from sklearn import metrics
```

```
from sklearn.ensemble import ExtraTreesClassifier #极端随机森林
```

```
model = ExtraTreesClassifier()
```

```
model.fit(X, y)
```

```
# 特征重要度
```

```
print(model.feature_importances_)
```

```
[0.1291864  0.26433718  
0.11710955 0.09038293  
0.08439176 0.16706915  
0.14752303]
```

- Pregnancies: 怀孕次数
- Glucose: 口服葡萄糖耐量试验2小时后血浆葡萄糖浓度
- BloodPressure: 舒张压 (mm Hg)
- SkinThickness: 三头肌皮褶厚度 (mm)
- Insulin: 餐后血清胰岛素水平 (mm)
- BMI: 体重指数 (体重(kg)/身高(m²))
- DiabetesPedigreeFunction: 糖尿病家系函数

4.建模与评估

```
from sklearn import metrics  
from sklearn.svm import SVC  
model=SVC(C=0.5,kernel='linear')  
model.fit(X, y)  
print(model)
```

预测

```
expected = y  
predicted = model.predict(X)
```

- Pregnancies: 怀孕次数
- Glucose: 口服葡萄糖耐量试验2小时后血浆葡萄糖浓度
- BloodPressure: 舒张压 (mm Hg)
- SkinThickness: 三头肌皮褶厚度 (mm)
- Insulin: 餐后血清胰岛素水平 (mm)
- BMI: 体重指数 (体重(kg)/身高(m²))
- DiabetesPedigreeFunction: 糖尿病家系函数
- Age: 年龄
- Outcome: 标签 (0不患病/1患病) 1 4

输出评估结果

```
print('RESULT')
```

```
print(metrics.classification_report(expected, predicted))
```

```
print('CONFUSION MATRIX')
```

```
print(metrics.confusion_matrix(expected, predicted))
```

	precision	recall	f1-score	support
0.0	0.79	0.89	0.84	500
1.0	0.73	0.56	0.64	268
accuracy			0.77	768
macro avg	0.76	0.73	0.74	768
weighted avg	0.77	0.77	0.77	768

CONFUSION MATRIX

```
[[444  56]  
 [117 151]]
```

5.超参数调优

```
from sklearn.model_selection import GridSearchCV
params = {"C": [0.1,0.5,1.0, 5.0],"kernel":['linear','rbf']}
clf = SVC(decision_function_shape='ovr')
grid_search=GridSearchCV(clf, params, scoring="f1_macro", cv=5)
grid_result=grid_search.fit(X, y)#用训练数据进行网格搜索出模型的最佳超参数
print("最佳结果",grid_result.best_score_)
print("最佳参数",grid_result.best_params_)
print("最佳估计器",grid_result.best_estimator_)
```

最佳结果 0.738280854992273

最佳参数 {'C': 5.0, 'kernel': 'linear'}

最佳估计器 SVC(C=5.0, kernel='linear')

➤ 准备数据

- ✓ 读取数据，提取特征；
- ✓ 将数据分割为训练集和测试集

➤ 配置模型

➤ 训练模型

➤ 模型评估

- ✓ 计算模型的准确率/精度
- ✓ 决策函数的决策之/预测值

➤ 模型预测

```
#1. 数据准备
#1.1 加载数据
data = np.loadtxt('/home/aistudio/data/data2301/iris.data',
                  dtype=float,
                  delimiter=',',
                  converters={4:iris_type})

#1.2 数据分割
x, y = np.split(data, (4,), axis=1)
x = x[:, :2]
x_train, x_test, y_train, y_test=model_selection.train_test_split(x,
                                                                    y,
                                                                    random_state=1,
                                                                    test_size=0.3)

#2. 定义模型: SVM模型定义
clf=classifier()

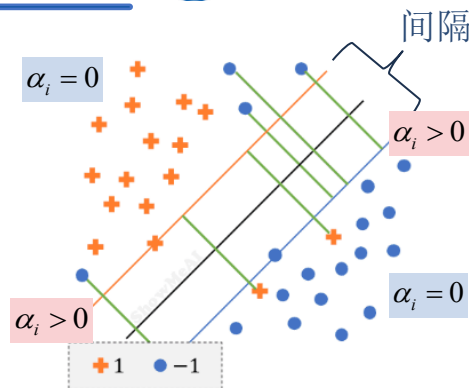
#3. 训练模型
train(clf,x_train,y_train)

#4. 模型评估
print_accuracy(clf,x_train,y_train,x_test,y_test)

#5. 模型使用
draw(clf,x)
```


支持向量机：一种**有监督学习**方法，尝试寻找一个**最优决策边界**，使**距离两个类别最近的样本最远**，从而对分类问题提供良好的泛化能力。

- » 支持向量机的“最大间隔”思想
- » 通过向高维空间映射解决线性不可分的问题
- » 引入“软间隔”缓解特征空间中线性不可分的问题
- » 将支持向量应用到回归问题上得到支持向量回归
- » 将核方法推广到其他学习模型



软间隔支持向量机

$\alpha_i > 0$ 的样本是支持向量

$0 < \alpha_i < C$ 的样本在间隔边界上

$\alpha_i = C$ 的样本可能被误分或在间隔内

C: 错误项的惩罚参数，它还控制平滑决策边界和正确分类训练点之间的权衡

以下哪项是支持向量机（SVM）的基本目标？

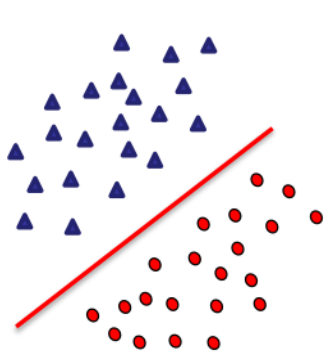
- ☐ A 最大化类别间隔
- ☐ B 最小化分类错误
- ☐ C 最大化分类错误
- ☐ D 最小化类别间隔

提交

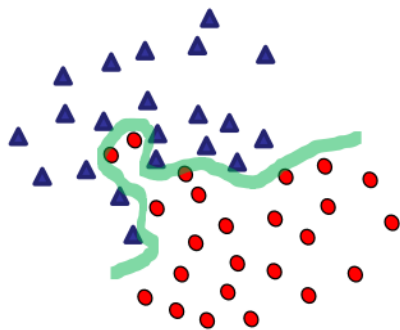
以下哪项是支持向量机（SVM）的基本目标？

- ☒ A 最大化类别间隔
- ☐ B 最小化分类错误
- ☐ C 最大化分类错误
- ☐ D 最小化类别间隔

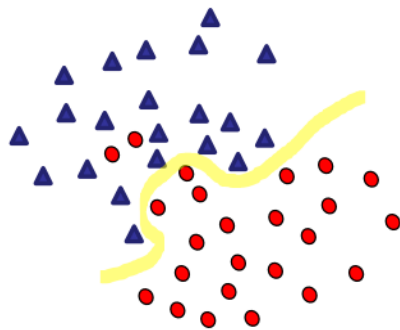
提交



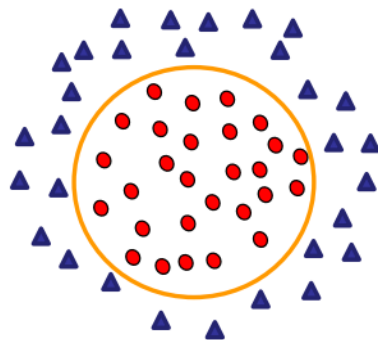
线性可分



硬间隔



软间隔



线性不可分

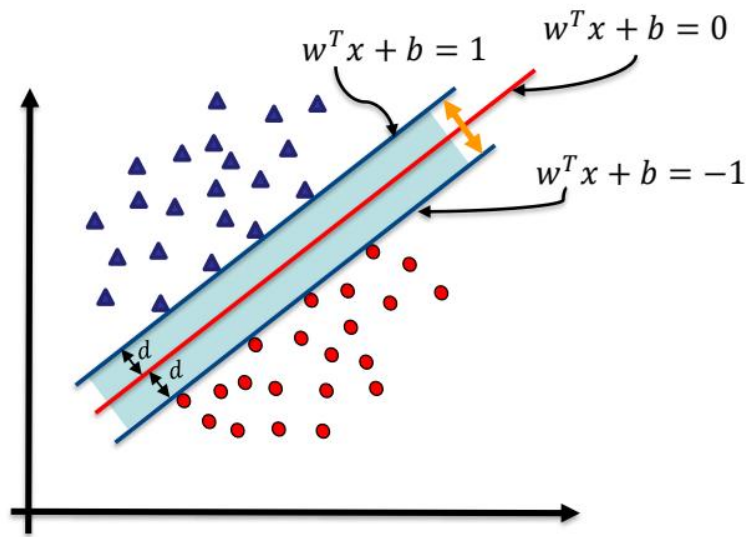
假如数据是完全的线性可分的，那么学习到的模型可以称为**硬间隔支持向量机**。换个说法，**硬间隔**指的就是**完全分类准确**，不能存在分类错误的情况。**软间隔**，就是**允许一定量的样本分类错误**。

n 为特征数， m 为训练样本数。

(1)如果相较于 m 而言， n 要大许多，即训练集数据量不够支持我们训练一个复杂的非线性模型，我们选用逻辑回归模型或者不带核函数的支持向量机。

(2)如果 n 较小，而且 m 大小中等，例如 n 在 1-1000 之间，而 m 在10-1000之间，使用高斯核函数的支持向量机。

(3)如果 n 较小，而 m 较大，例如 n 在1-1000之间，而 m 大于50000，则使用支持向量机会非常慢，解决方案是创造、增加更多的特征，然后使用逻辑回归或不带核函数的支持向量机。





1. 在支持向量机中，核函数的作用是什么，有哪些常用的核函数（至少列举3个核函数）？正则化参数 C 的作用是什么？
2. 试分析支持向量机SVM和逻辑回归LR之间的区别（至少列举4个不同点）。
3. 对带有高斯核（RBF核）和软间隔的SVM，引入拉格朗日乘子后，给出构建的拉格朗日函数。